7 Written Problems:

Abstraction:

1. It lacks some specificity on preconditions, postconditions, and what the input values and return values are.

```
/**
Adds x to the set if not already there, in sorted order
Precondition: not null
@param x element added to set
*/
void add(T x){};
/**
* Tests element if it is in the set
* @param x element to be tested if in set
* @return true if in the sorted set
*/
boolean contains (T x){};
/**
* Removes first instance of x if in the list
* @param x element to be searched
*/
void remove(T x){};
/**
* @return first element in sorted set, null if set is empty
*/
T first(){};
```

2.

```
public boolean remove(T key){
if (!isEmpty()) {
   ListNode<T> tracker = head;
   ListNode<T> behindtracker = null;
   while (tracker != null) {
       if (key.compareTo(tracker) == 0) {
           if (tracker == head) {
               head = tracker.next;
               if (sizelist == 1) {
                   tail = null;
               }
           } else if (tracker == tail) {
               tail = behindtracker;
               tail.next = null;
```

```
            } else {
                behindtracker.next = tracker.next;
            }
            sizelist--;
            return true;
        }
        behindtracker = tracker;
        tracker = tracker.next;
    }
}
return false;
}

public T first(){
    if(head==null){
        return null;
    }
    return head.value;
}
```

3.

```
public void prepend(K key, V value) {
    Node<K, V> h = new Node<>(key, value, head);
    head = h;
    if (isEmpty()) {
        tail = h;
    }
    sizelist++;
}


public V remove(K key) {
    if (!isEmpty()) {
        Node<K, V> tracker = head;
        Node<K, V> behindtracker = null;
        while (tracker != null) {
            if (tracker.key.equals(key)) {
                if (tracker == head) {
                    head = tracker.next;
                    if (sizelist == 1) {
                        tail = null;
                    }
                } else if (tracker == tail) {
                    tail = behindtracker;
```

```
                    tail.next = null;
                } else {
                    behindtracker.next = tracker.next;
                }
                sizelist--;
                return tracker.value;
            }
            behindtracker = tracker;
            tracker = tracker.next;
        }
    }
    return null;
}

public T first(){
    if(head==null){
        return null;
    }
    return head.value;
}
```

4. It would be more appropriate to go for an unsorted list if all you care about is fast insertions and are constantly modifying the list, and use a sorted list if you care about searching for elements inside a list, but don't often insert into it. This is because a sorted list has lg n lookup time for searching, but n time to insert (to preserve sortedness), while an unsorted list has n lookup time because it has to manually traverse it, while it has O(1) insertion because it just appends to the start without care for order.

Asymptotic Complexity:

5. $O(n^2)$

It is $O(n^2)$ because it has an outer loop of length n-5, and within that loop, it will about half the time execute a loop of n-i-1, which then prints something about 70,000 times.

$(n-5)*(½ * n-i-1)*(69993) = O(n^2)$ by dominant term and removing constants

$½ n^2 = O(n^2)$

6.

I'm going to assume log base 2, but base 10 is workable as well, but makes it even stronger

$n^2 log(n) = kn^3$

Divide n^2 on both sides

$log(n) = kn$

k = 1/2, n = 4

$\frac{1}{ln2 * x} \leq 1/2,\ x >= 4,\ 1/ln2 * 4 = 1/2.77$

From here, the derivatives for both, 1/ln2*x is below 1/2, the deriv for the other one, when x >= 4, so the kn must be larger from that point, achieving equality at the witness pair, and never meeting paths again.

7.

$f_1(n) \leq c \bullet n^2$

$f_2(n) \leq b \bullet n^2$

$f_1(n) + f_2(n) \leq O(n^2)$

$f_1(n) + f_2(n) \leq kn^2$

K = c+b, n_0 = 0;

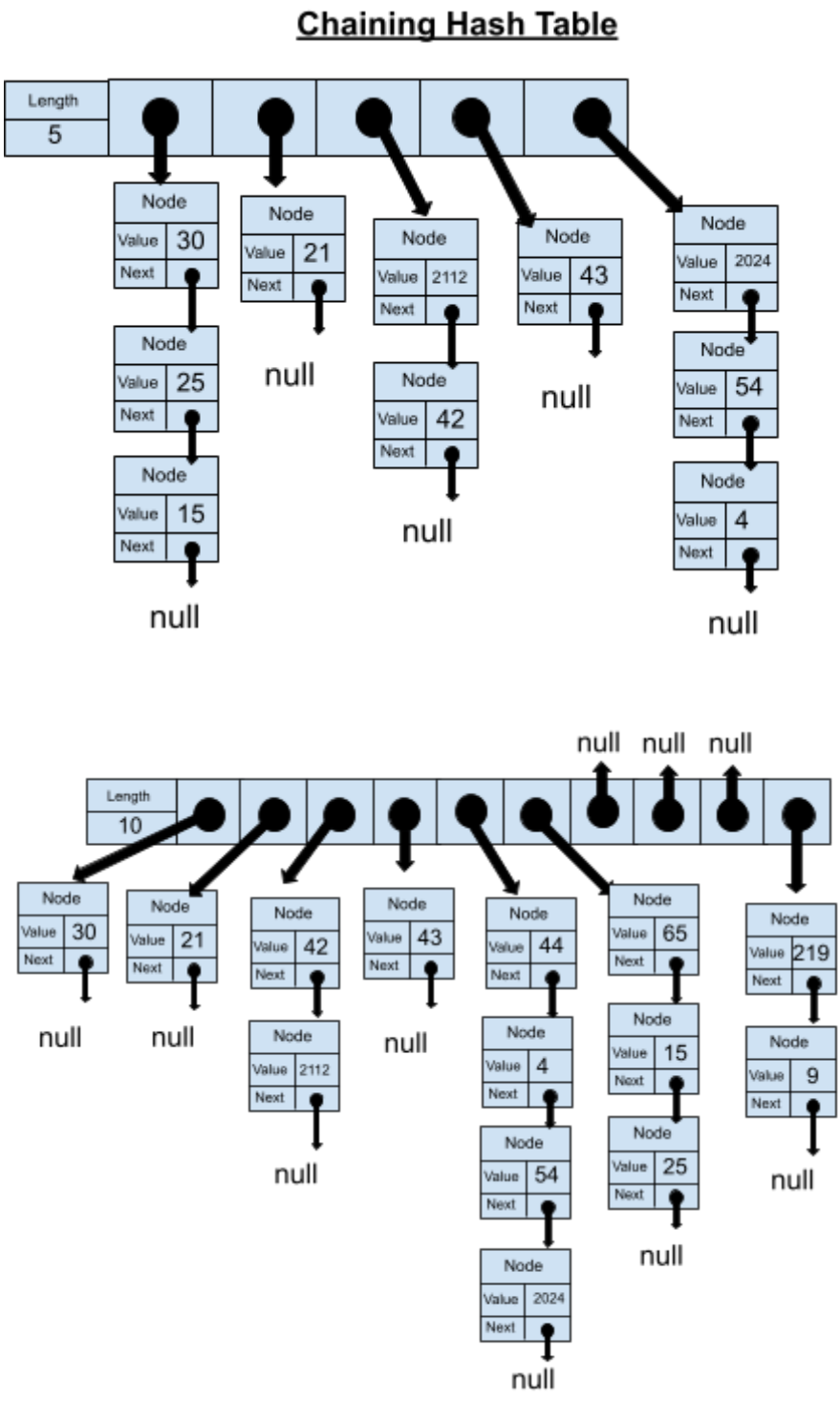
$(c + b) \bullet n^2 \leq (c + b) \bullet n^2$

8.

No such witness exists, because $3^{2n} \leq k3^n$ when turned into a ratio test/limit test, is $3^n \leq k$, which can't be true, because the $\lim_{n \to \infty} 3^n = \infty$, which is not a real number.

Hashing:

9.


Chaining Hash Table
Length
5
Node
Value
Next
30
25
15
21
2112
42
43
2024
54
4
null
Length
10
65
44
4
54
2024
15
25
219
9

Linear Probing Hash Table